

Earthquake Co-Occurrence Analysis with Apache Spark

Scalable and Cloud Programming, Project Report, A.Y. 2025/2026

Matteo Fontana, University of Bologna

Source code: <https://github.com/ma3ti/Earthquake-Co-Occurrence-Analysis-with-Apache-Spark>

1 Problem and dataset

The goal of the project is to find, in a large catalogue of earthquake events, the pair of geographic locations whose earthquakes co-occur most frequently on the same day, and to print that pair together with the list of its co-occurrence dates in ascending order. Following the assignment specification, the location of an event is discretised by rounding latitude and longitude to the first decimal digit (half-up rounding), so that each rounded pair identifies a cell of roughly 11×9 km at mid latitudes. The time of an event is discretised to its calendar day. Duplicate events, that is multiple earthquakes in the same cell on the same day, must be counted once, otherwise the most frequent “pair” would degenerate into a cell paired with itself.

The full dataset contains 3 445 751 events (about 180 MB of CSV) with header `longitude,latitude,date`, covering the years 1990 to 2023. Columns are read by name, so their order is irrelevant; the day is extracted as the first ten characters of the timestamp. All Google Cloud experiments use the full dataset, as required.

2 Approach: two MapReduce implementations

Both implementations are pure RDD (MapReduce) pipelines that share the same preprocessing, accept the same arguments (the dataset URI and an optional number of partitions, applied with `repartition`) and print identical output, so that their performance can be compared directly.

2.1 Common preprocessing

Each CSV row is mapped to a pair `(day, (lat, lon))`, with coordinates rounded via `BigDecimal(...).setScale(1, HALF_UP)` and the day taken as `date.substring(0,10)`.

2.2 V1: self-join

V1 is the direct translation of the problem statement into MapReduce:

1. Deduplicate: `distinct()` keeps one event per `(day, cell)`.
2. Self-join on the day: `parsed.join(parsed)` generates, for every day, all ordered pairs of cells active on that day.
3. Filter: only pairs with `loc1 < loc2` (lexicographic on the string form) are kept, discarding self-pairs `(A,A)` and one of each symmetric pair `(A,B)/(B,A)`.
4. Aggregate: each surviving pair is mapped to `((loc1,loc2), List(day))` and the date lists are concatenated with `reduceByKey(_ ++ _)`.
5. Select: `sortBy(_._2.size, desc).first()` returns the winning pair; its dates are sorted and printed.

V1 is correct but has three structural inefficiencies. For a day with k active cells the join materialises k^2 pairs before the filter discards more than half of them; the aggregation drags entire date lists through the shuffle, with a payload that grows as counts accumulate; and a global sort (one extra shuffle, preceded by a sampling job for the range partitioner) is used merely to select a maximum. The pipeline performs four shuffles: `distinct`, `join`, `reduceByKey`, `sortBy`.

2.3 V2: per-day combinations

V2 removes all three inefficiencies by never joining the dataset with itself:

1. Group and deduplicate in one shuffle: `aggregateByKey(Set)` builds, per day, the set of active cells, and the set semantics subsume `distinct()`. The resulting RDD has one record per distinct day (order of 10^4 records, more than two orders of magnitude smaller than the input) and is cached with `persist(MEMORY_AND_DISK)` for reuse.
2. Generate each pair exactly once: per day, the cell set is sorted with the same lexicographic key used by V1 and streamed through `combinations(2)`, a lazy iterator that yields each of the $\binom{k}{2}$ pairs once, already ordered. There are no symmetric pairs, no self-pairs, no post-filter, and no materialisation of the pair set in memory.
3. Count, do not collect: pairs are counted with `reduceByKey(_ + _)` on integers, which benefits from map-side combining. The shuffle carries one `Int` per distinct pair per map task instead of growing date lists.

4. Maximum via **reduce**: per-partition maxima are combined in a single action, with no global sort and no sampling job.
5. Recover the dates of the winner: the cached per-day RDD is filtered for the days whose set contains both winning cells ($O(1)$ set lookups), and the roughly 10k resulting date strings are collected and sorted on the driver.

V2 performs two shuffles (**aggregateByKey**, **reduceByKey**). Table 1 summarises the differences.

	V1 (self-join)	V2 (combinations)
Shuffles	4 (distinct , join , reduceByKey , sortBy)	2 (aggregateByKey , reduceByKey)
Pairs generated per day (k cells)	k^2 , filtered to $\binom{k}{2}$ afterwards	$\binom{k}{2}$, streamed lazily
Shuffle payload of the aggregation	concatenated date Lists	Int counts, map-side combined
Finding the maximum	global sort and sampling job, then first()	single reduce
Dates of the winning pair	carried through the whole pipeline	filter on the cached per-day RDD

Table 1: Structural comparison of the two implementations.

3 Experimental setup

All experiments ran on Google Cloud Dataproc (region **europe-west1**) with one master and 2, 3 or 4 workers, all **n2-standard-4** machines (4 vCPUs, 16 GB RAM), the type required by the assignment, with 240 GB boot disks. The application is packaged as a single fat JAR (sbt-assembly, Scala 2.12, Spark dependencies provided by the Dataproc runtime) containing both classes; dataset and JAR reside in a Cloud Storage bucket.

Each cluster size was tested with Spark’s default partitioning and with explicit **repartition** values chosen as multiples of the total number of worker cores, from $0.5\times$ to $8\times$. The full matrix (15 V1 and 9 V2 configurations) was executed with shell scripts, committed in the repository, that for every cluster size create the cluster, submit the jobs and delete the cluster, so that the runs are reproducible and no cluster is left running. The reported metric is the job wall-clock time from the Dataproc console (a screenshot of the complete job list is committed as evidence, together with the raw driver output of one run). It includes the application start-up, a fixed cost of the order of a minute that is the same in every configuration. Because of the education-credit budget each configuration was run once, so run-to-run variance is not measured; the one observation we attribute to it is flagged below.

4 Result of the analysis

Every completed run (both algorithms, every cluster size and partitioning) produced the identical result:

$((38.8, -122.7), (38.8, -122.8))$ with 10014 dates from 1990-01-05 to 2023-07-29.

The date list was verified to be strictly ascending and free of duplicates on the committed driver output. The agreement of two algorithmically independent implementations on all 21 successful runs is strong evidence of correctness.

The result is also geophysically meaningful. The two adjacent cells cover The Geysers geothermal field in Northern California, the largest geothermal complex in the world, where fluid injection induces small earthquakes essentially every day. The pair co-occurs on 82% of all days in the 33-year window, and the density increases over time: 62% of the days in the 1990s, 88% in the 2000s, 91% in the 2010s and 93% from 2020 to 2023. This trend is plausible given the expansion of injection operations at the field, although improvements in the sensitivity of the seismic network may also contribute. In any case it is an independent sanity check that the pipeline measures a real phenomenon rather than an artefact.

5 Performance and scalability analysis

Table 2 reports the complete experimental matrix; Figures 1 and 2 visualise it.

5.1 Default partitioning: the cluster does not matter if the job cannot use it

The 180 MB CSV is split by Spark into about two input partitions (128 MB being the default maximum split size), and this throttles the whole pipeline. In V2 the map side of the first shuffle, which parses 3.4 M rows and pre-aggregates them, runs as very few tasks regardless of the cluster and dominates the runtime. The default-partitioning runs take 27 m 03 s, 26 m 08 s and 26 m 34 s on 2, 3 and 4 workers respectively, which is flat. Paying for twice the hardware buys nothing, because only a few of the 8 to 16 cores ever work.

V1 fares even worse. The join and the aggregation downstream of **distinct()** keep the same small number of partitions as the input, so even the quadratic join runs as a few very large tasks. Such tasks are likely to

Workers	Cores	Partitions	V1 (self-join)	V2 (combinations)	V2 speedup
4	16	default	1 h 16 m	26 m 34 s	2.9×
		8	39 m 40 s	not run	
		16	24 m 33 s	6 m 46 s	3.6×
		32	14 m 45 s	6 m 45 s	2.2×
		64	10 m 07 s	not run	
		128	8 m 57 s	not run	
3	12	default	49 m 26 s	26 m 08 s	1.9×
		12	failed (OOM)	8 m 04 s	completes
		24	19 m 16 s	8 m 01 s	2.4×
		48	13 m 45 s	not run	
2	8	default	37 m 26 s	27 m 03 s	1.4×
		8	failed (OOM)	28 m 39 s	completes
		16	failed (OOM)	16 m 17 s	completes
		32	25 m 29 s	not run	
		64	19 m 28 s	not run	

Table 2: Wall-clock job time for every configuration (full dataset, **n2-standard-4** workers).

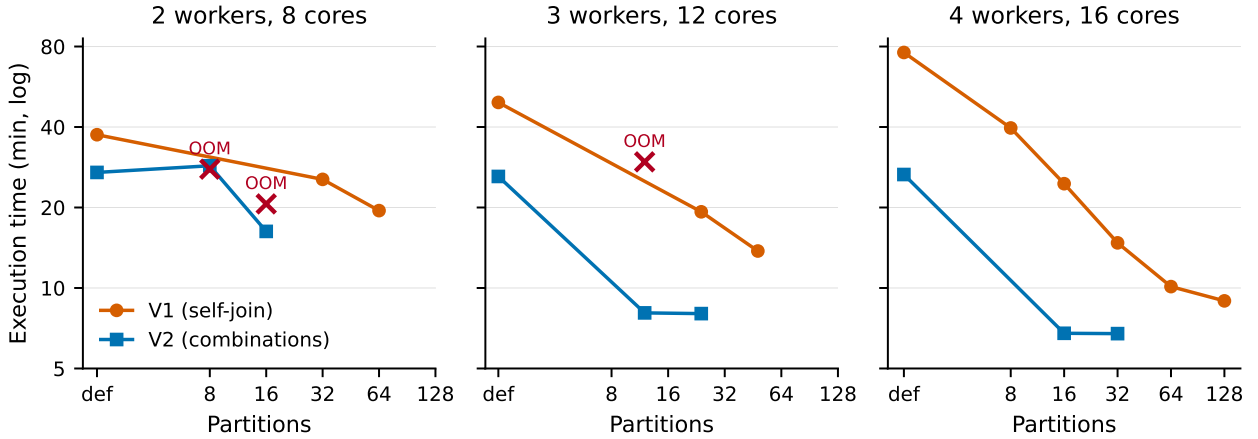


Figure 1: Execution time vs. number of partitions (log-log), one panel per cluster. The default partitioning is drawn at $x = 2$ as a nominal position, since the input file is split into very few partitions. Red crosses mark V1 runs aborted by executor OOM, placed at the time of abort.

operate at the edge of memory and to spill heavily, which makes them both slow and unstable: 37, 49 and 76 minutes on 2, 3 and 4 workers. We attribute this inversion (more workers, slower) to run-to-run variance of such a memory-stressed computation rather than to a systematic cause. With a single run per configuration the three values should be read as between about 40 and 75 minutes, presumably dominated by spill and garbage-collection behaviour, against the stable 26.5 minutes of V2.

Explicit repartitioning removes the ceiling and is by far the largest single factor observed: on 4 workers it improves V1 by $8.5\times$ (1 h 16 m to 8 m 57 s) and V2 by $3.9\times$ (26 m 34 s to 6 m 45 s).

5.2 The V1 failures are algorithmic, not a cluster limit

Three V1 configurations (3w-12p, 2w-8p, 2w-16p) failed with executors repeatedly killed with exit code 137 (SIGKILL, reported as “Killed by external signal”), the typical signature of an out-of-memory kill, until a task exhausted its four attempts and the job aborted. The failures are consistent with memory pressure. They occur when partitions are coarse (1 to 2 per core), so that each join task materialises the k^2 pair explosion and the growing date lists for a large slice of days, and the cluster is small, so that several such tasks run concurrently on each node and share its memory. Finer partitions shrink the per-task working set (2w-32p and 2w-64p complete), and on the 4-worker cluster even 8 coarse partitions survive, since each node hosts only about two such tasks instead of four.

The decisive evidence that the limit is algorithmic comes from V2: all three failing configurations complete under V2 on identical clusters (8 m 04 s, 28 m 39 s and 16 m 17 s). Streaming pair generation and integer counters keep the per-task working set small no matter how coarse the partitioning. For a practitioner this is the more important property: V2 is not only faster, it is robust across the configuration space, while V1 requires tuning merely to terminate.

5.3 Partition sensitivity: memory-bound vs. compute-bound

Figure 1 shows two different response curves. V1 keeps improving up to 8 partitions per core (4 workers: 39.7, 24.6, 14.8, 10.1 and 9.0 minutes for 8, 16, 32, 64 and 128 partitions). Extra partitions most likely relieve memory pressure (smaller join state, less spill), so the returns diminish slowly. V2 plateaus at about one partition per core: on 4 workers, 16 and 32 partitions are indistinguishable (6 m 46 s vs. 6 m 45 s), and on 3 workers 12 and 24 partitions likewise (about 8 min). Once every core has work, V2 is compute-bound in the pair-generation stage and additional partitions only add scheduling overhead. The one V2 outlier is 2w-8p (28 m 39 s, no better than default). With one coarse partition per core on the smallest memory budget, the map-side combiners of the pair-count stage probably spill, and the tasks that receive several seismically busy days become stragglers with no idle cores to absorb them; 16 partitions fix both effects (16 m 17 s).

A striking consequence of the plateau is that V2 with 16 partitions (6 m 46 s) beats V1 with its best configuration of 128 partitions (8 m 57 s). A better algorithm outperforms brute-force tuning of a worse one, with fewer and lighter shuffles.

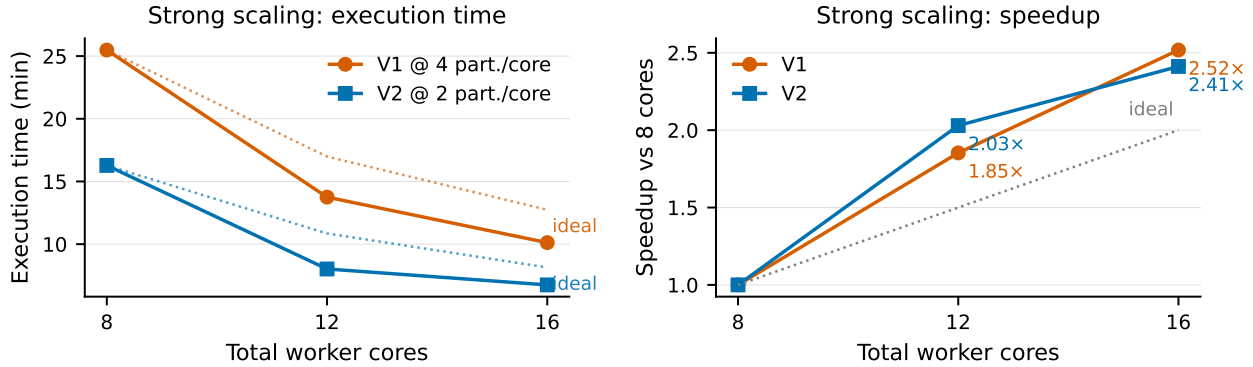


Figure 2: Strong scaling at fixed partitions per core (V1 at 4 per core: 32, 48 and 64 partitions; V2 at 2 per core: 16, 24 and 32 partitions). Dotted lines show ideal scaling from the 8-core baseline.

5.4 Strong scaling

Figure 2 isolates cluster-size scaling by holding the number of partitions per core constant. Both versions scale well, and superlinearly on the first step: going from 8 to 12 cores (1.5× the resources) yields speedups of 1.85× for V1 and 2.03× for V2. Superlinearity is plausible here, because the aggregate cluster memory grows with the worker count: shuffle and aggregation state that spilled to disk on 2 workers can stay in memory on 3, removing I/O that the ideal-scaling model does not account for. Over the whole range from 8 to 16 cores the cumulative speedups are 2.52× for V1 and 2.41× for V2, both above the ideal 2×.

The marginal step from 12 to 16 cores (1.33× the resources) is where the two versions differ. V1 still gains 1.36× (13.8 to 10.1 minutes), whereas V2 gains only 1.19× (8.0 to 6.8 minutes). V2 is already likely to be close to the floor set by the fixed start-up cost of about a minute and by data skew: The Geysers region makes some days far heavier than others, and the pair generation for the busiest days executes inside single tasks that extra workers cannot subdivide. For this workload, 3 workers with correct partitioning (8 minutes) are the resource-efficiency sweet spot for V2, since the fourth worker buys only 1.3 minutes.

6 Conclusions

Both implementations solve the assignment on the full dataset and agree, across 21 successful runs, on a geophysically plausible result (The Geysers cell pair, 10014 dates). The performance study supports three conclusions. First, partitioning dominates: for a small file the default input parallelism idles the cluster, and no hardware can compensate for it (V2 with default partitioning is flat at about 26.5 minutes on every cluster). Second, the algorithm matters more than the hardware: replacing the self-join with per-day streamed combinations cuts the shuffles from 4 to 2, gives speedups between 1.4× and 3.6× at equal configuration, lets a mid-size configuration beat the best brute-force tuning of V1, and, most importantly, turns three OOM-failing configurations into successes on identical clusters. Third, scaling is real but bounded: it is near-ideal, even superlinear thanks to reduced spill, up to 12 cores, and for V2 it flattens beyond that point because of the start-up overhead and the skew of the data. The best observed configuration is V2 on 4 workers with 32 partitions: 6 m 45 s, an 11.3× improvement over the naive algorithm with default partitioning on the same cluster.